SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: **Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.** ***(BL3)***

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration: 1 Hour**

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I K. Bhavya Sri/192 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Bhavya Sri

CONSTRAINT-BASED PROBLEM SOLVING

1. Doctor Shift Assignment Using Backtracking Search

1.1 Problem Statement

A hospital needs to assign three doctors (D1, D2, D3) to three shifts: Morning, Afternoon, and Night. Each doctor must be assigned exactly one shift while satisfying all the given constraints.

Constraints:

1. D1 cannot work the Night shift.
2. D2 must work before D3.
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must receive exactly one shift.

1.2 Algorithm: Backtracking Search

Backtracking Search assigns values to variables one at a time. After every assignment, the constraints are checked. If an assignment violates any constraint, it is rejected, and the algorithm tries another value.

Steps:

1. Start with an empty schedule.
2. Assign a shift to D1.
3. Check whether D1's shift satisfies the constraints.
4. Assign a different shift to D2.
5. Ensure D2 works before D3.
6. Assign a shift to D3.
7. Check all constraints.
8. If all constraints are satisfied, return the final schedule. Otherwise, backtrack and try another assignment.

1.3 Intermediate Steps and Constraint Checks

The doctors are assigned in the order D1 → D2 → D3. Shifts are tried in the order Morning → Afternoon → Night.

Step	Assignment Attempt	Constraint Check	Decision

1	D1 = Morning	D1 is not assigned Night.	Accept
2	D2 = Morning	Morning is already assigned to D1.	Reject
3	D2 = Afternoon	Different shift from D1; D2 can work before D3.	Accept
4	D3 = Morning	D3 cannot work Morning.	Reject
5	D3 = Afternoon	Afternoon is already assigned to D2.	Reject
6	D3 = Night	D3 is not on Morning, and D2 works before D3.	Accept

Backtracking explanation: When D2 is assigned Morning, the algorithm detects a shift conflict and rejects that assignment. It then tries Afternoon. Similarly, invalid assignments for D3 are rejected until Night satisfies all constraints.

1.4 Final Valid Schedule

Shift	Assigned Doctor
Morning	D1
Afternoon	D2
Night	D3

1.5 Verification

- D1 works Morning, not Night.
- D2 works Afternoon, before D3 on Night.
- Each shift has exactly one doctor.
- D3 does not work Morning.
- Every doctor has exactly one shift.

Result: Backtracking Search successfully finds the valid schedule: Morning – D1, Afternoon – D2, Night – D3.

2. Grid Pathfinding Using Search Algorithms

2.1 Problem Statement

A robot operates in a 5×5 grid and must move from the Start (S) to the Goal (G). Some cells contain obstacles that the robot cannot cross.

The objective is to find the shortest path from the start to the goal while avoiding obstacles.

Given constraints:

1. Movement is allowed Up, Down, Left, and Right.
2. Each movement has a cost of 1.
3. The robot cannot pass through obstacles.
4. Manhattan Distance is used as a heuristic.
5. The optimal path and total cost must be determined.

2.2 Grid Representation

Since the question does not specify the exact obstacle positions, the following example grid is used.

Row / Column	**1**	**2**	**3**	**4**	**5**
1	S	.	#	.	.
2	.	.	#	.	#
3	#	.	.	.	.
4	.	#	#	.	.
5	.	.	.	#	G

Legend:

- S – Start position (1,1)
- G – Goal position (5,5)
- **– Obstacle**
- . – Free cell

2.3 Breadth-First Search (BFS) Algorithm

Breadth-First Search explores nodes level by level. It uses a FIFO queue (First In, First Out) to store the nodes waiting to be explored.

Algorithm steps:

1. Place the start node S in the queue.
2. Remove the first node from the queue.
3. Check whether the current node is the goal.
4. Generate its neighboring cells in the order Up, Down, Left, Right.
5. Reject cells that are outside the grid, blocked, or already visited.
6. Add valid, unvisited neighbors to the queue and record their parent nodes.
7. Repeat until the goal is reached or the queue becomes empty.

8. Reconstruct the path by following the parent nodes from G back to S.

2.4 Step-by-Step Node Expansion and Queue Updates

The queue below is shown from front → rear. Each node is represented by its row and column coordinates.

Step	Expanded Node	New Valid Nodes Added	Queue After Expansion
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(2,2)	(1,2), (2,2)
3	(1,2)	None	(2,2)
4	(2,2)	(3,2)	(3,2)
5	(3,2)	(3,3)	(3,3)
6	(3,3)	(3,4)	(3,4)
7	(3,4)	(2,4), (4,4), (3,5)	(2,4), (4,4), (3,5)
8	(2,4)	(1,4)	(4,4), (3,5), (1,4)
9	(4,4)	(4,5)	(3,5), (1,4), (4,5)
10	(3,5)	None	(1,4), (4,5)
11	(1,4)	(1,5)	(4,5), (1,5)
12	(4,5)	(5,5) – Goal	(1,5), (5,5)
13	(1,5)	None	(5,5)
14	(5,5)	Goal reached	Empty

Explanation of constraint checks:

- Obstacles such as (1,3), (2,3), and (5,4) are never entered.
- Previously visited cells are not added to the queue again.
- Only valid neighboring cells are expanded.
- When the goal (5,5) is reached, the search stops.

2.5 Manhattan Distance Heuristic

Manhattan Distance estimates the distance between a current cell and the goal when movement is restricted to four directions.

The formula is:

$$ h(n)=|r-r_G|+|c-c_G| $$

Where:

- \(r,c\) are the current cell coordinates.
- \(r_G,c_G\) are the goal coordinates.

For the start cell (1,1) and goal cell (5,5):

\[h(S)=|1-5|+|1-5|=8 \]

The estimated distance is 8 moves.

Some example heuristic values are:

Cell	Manhattan Distance to G
(1,1)	8
(2,1)	7
(2,2)	6
(3,3)	4
(3,4)	3
(4,4)	2
(4,5)	1
(5,5)	0

The heuristic decreases as the robot approaches the goal along the selected shortest path.

2.6 Priority Queue and Heuristic Search

There is an important distinction between BFS and heuristic search:

- BFS: Uses a FIFO queue and does not use Manhattan Distance to prioritize nodes.
- A* Search: Uses a priority queue and combines the actual cost with the heuristic.

For A* Search, the evaluation function is:

\[f(n)=g(n)+h(n) \]

Where:

- \(g(n)\) = cost from the start to the current node.
- \(h(n)\) = estimated cost from the current node to the goal.
- \(f(n)\) = estimated total path cost.

For example, at node (2,2), the path cost is $g=2$, and the Manhattan distance is $h=6$.

$$ f(n)=2+6=8 $$

A* selects the frontier node with the lowest $f(n)$, using a consistent tie-breaking rule when necessary. This is the appropriate method if the assignment requires both Manhattan Distance and priority queue updates.

2.7 Optimal Path and Total Cost

One optimal path obtained using BFS is:

$$ \begin{aligned} (1,1)&\rightarrow(2,1)\rightarrow(2,2)\\ &\rightarrow(3,2)\rightarrow(3,3)\rightarrow(3,4)\\ &\rightarrow(4,4)\rightarrow(4,5)\rightarrow(5,5) \end{aligned} $$

The path contains 8 movements.

Since each movement costs 1:

$$ \text{Total Cost}=8\times1=\boxed{8} $$

Result: BFS finds a shortest path of 8 moves with a total cost of 8. Manhattan Distance can also guide A* Search toward the goal, and A* is optimal for this grid with unit movement costs.

3. Autonomous Rescue Robot in a Partially Observable Environment

3.1 Problem Statement

An autonomous rescue robot is deployed in a disaster-affected building to locate a survivor. The building is represented as a grid of rooms, with the robot starting at position S and the survivor located at position G.

The robot has limited sensing capability and can observe only adjacent cells. Some cells are blocked, and some cells are risky. The robot must navigate safely while minimizing the total movement cost.

Given constraints:

1. The robot can move Up, Down, Left, and Right.
2. Obstacles cannot be traversed.
3. The robot can observe only adjacent cells.
4. Each normal movement costs 1.
5. Entering a risky zone adds an extra cost of 2, making the total cost 3 for that move.
6. The robot must minimize total cost while ensuring safe navigation.

7. The robot must update its knowledge dynamically as it discovers new information.

3.2 (a) Analysis of Constraints and Their Effects

Constraint	Effect on Robot's Decision-Making
Four-direction movement	The robot can move only Up, Down, Left, or Right.
Obstacles	The robot must avoid blocked cells and find alternative routes.
Limited sensing	The robot cannot see the complete environment initially and must explore nearby cells.
Movement cost of 1	Every normal movement increases the total path cost by 1.
Risk-zone cost of 3	Entering a risky cell costs 1 for movement plus 2 extra units.
Cost minimization	The robot must compare possible routes and choose the least-cost feasible route.
Dynamic knowledge	The robot must update its internal map whenever new information is obtained.
Safety requirement	The robot should avoid dangerous routes and prefer safer alternatives when feasible.

Explanation:

The robot cannot plan perfectly using a complete map because it does not know the condition of every cell. It must sense its surroundings, identify obstacles, and determine whether nearby cells are safe or risky.

A route with fewer movements may not be the cheapest if it passes through risky zones. Therefore, the robot must consider both movement costs and safety information before selecting a route.

3.3 (b) Proposed Solution Using Uniform Cost Search and Online Search

Uniform Cost Search (UCS) is suitable because different movements can have different costs. It expands the available node with the lowest accumulated path cost.

The robot can combine UCS with online search: it plans using its currently known map, moves, senses its surroundings, updates the map, and replans when necessary.

Algorithm Steps

1. Initialize the robot at the starting position S.
2. Sense the adjacent cells and identify free, blocked, and risky cells.

3. Store the known information in an internal map.
4. Add S to a priority queue with a path cost of 0.
5. Remove the node with the lowest accumulated cost from the priority queue.
6. Check whether the selected node is the goal G. If yes, reconstruct the path and stop.
7. Examine the four neighboring cells.
8. Ignore blocked cells and calculate the cost of each valid movement.
9. For a normal cell, add 1 to the current cost. For a risky cell, add 3.
10. Update the cost and parent of a neighboring cell if a cheaper route is found.
11. Move toward the next selected cell and sense the surroundings again.
12. Update the internal map and replan if newly discovered obstacles or risks affect the route.
13. Continue until the robot reaches the survivor or determines that no feasible safe route is known.

Pseudocode

```
BEGIN
1. Set current position = S
2. Sense adjacent cells and update the known map
3. WHILE current position is not G:
    Apply Uniform Cost Search on the known map
    IF no feasible route is known:
        Sense surroundings
        Update the map
        Replan
    Select the next cell on the lowest-cost route
    Move to that cell
    Sense adjacent cells
    Update the map
4. IF current position = G:
    Display the path and total cost
```

END

Cost Calculation

The movement cost is calculated as:

$$\text{Move Cost}= \begin{cases} 1, & \text{Normal cell}\\ 3, & \text{Risky cell} \end{cases}$$

The total path cost is:

$$\text{Total Cost}=\sum \text{Movement Costs}$$

For example, suppose a route contains 5 normal movements and 2 risky-cell entries.

$$\text{Total Cost}=5(1)+2(3)=11$$

Therefore, the total cost of this route is 11.

3.4 Illustrative Route Cost Comparison

The following table is an example to demonstrate how the robot compares routes. These are hypothetical routes, not calculated paths through a specified building grid.

Route	Normal Moves	Risky Cells	Total Cost
Route A	6	1	$6+3=9$
Route B	8	0	$8+0=8$
Route C	7	0	$7+0=7$

In this example, Route C is the cheapest route, with a total cost of 7.

The robot should select Route C if it is feasible and safe. This demonstrates that the route with the fewest movements is not necessarily the best route when risk and movement costs differ.

Note: An exact rescue path and node-by-node search trace require the actual grid, start and goal coordinates, obstacle locations, and risk-zone locations.

3.5 (c) AI Concepts Applied to the Rescue Robot

1. Intelligent Agent

An intelligent agent perceives its environment, processes information, and takes actions to achieve a goal.

In this scenario, the robot senses adjacent rooms, identifies obstacles and risks, selects movements, and attempts to reach the survivor.

2. Rationality

A rational agent chooses actions that best achieve its objective based on the information currently available.

The robot compares route costs, avoids blocked cells, considers risky zones, and selects the least-cost feasible route while respecting safety requirements.

3. Partially Observable Environment

The environment is partially observable because the robot can sense only adjacent cells. It does not initially know the complete building layout.

The robot must explore the environment and update its internal map as it receives new information.

4. Sequential Environment

The environment is sequential because each movement affects the robot's current position and its future choices. Every action can influence the remaining route and total cost.

5. Dynamic Knowledge

The robot's knowledge changes as it explores. It may discover a previously unknown obstacle or risky area and need to change its plan.

6. Goal-Based Agent

The robot has a specific goal: reach the survivor at position G. Its actions are selected according to whether they help it reach that goal safely and efficiently.

7. Cost-Based Decision-Making

The robot considers the cost of each movement and the additional cost of risky cells. It selects a route with the lowest total cost among the feasible routes known to it.

3.6 (d) Justification for the Chosen Approach

Uniform Cost Search combined with online sensing is appropriate for the following reasons:

1. Different movement costs: UCS handles normal and risky movements with different costs.
2. Cost optimization: It expands the lowest-cost frontier node first and finds a least-cost path when the map and costs are known.
3. Partial observability: Online sensing allows the robot to discover its surroundings gradually.
4. Dynamic replanning: The robot can update its map and calculate a new route when it discovers obstacles or risks.
5. Safety awareness: The cost model discourages routes through risky cells when a cheaper alternative exists.

6. Systematic exploration: The priority queue helps the robot organize possible routes according to their accumulated cost.

Important safety consideration: Assigning a finite cost to risky cells does not guarantee that the robot will always avoid danger. Confirmed dangerous areas should be treated as forbidden whenever required by the safety policy. The robot should request assistance if no safe route is available.

3.7 Limitations

- The exact route cannot be calculated without the building grid and obstacle and risk-zone locations.
- Incorrect sensor readings may lead to unsuitable decisions.
- Newly discovered obstacles may require repeated planning.
- UCS is optimal for a known map with positive movement costs, but online planning may need to be repeated as new information becomes available.
- Real rescue systems require additional safety measures and communication with emergency responders.

Conclusion

Constraint-based problem solving enables AI systems to find valid solutions while satisfying specified conditions.

In Question 1, Backtracking Search finds a valid doctor schedule by rejecting assignments that violate constraints.

In Question 2, BFS finds the shortest path in a grid when all movements have equal cost. Manhattan Distance is a heuristic used by A* Search, which combines the path cost and estimated distance to the goal.

In Question 3, Uniform Cost Search combined with online sensing enables a rescue robot to navigate using movement costs, obstacle information, and risk assessments. The robot updates its knowledge and replans when new information becomes available.

These methods demonstrate how AI search algorithms support scheduling, pathfinding, decision-making, and safe autonomous navigation.